SCC.439 Programming Assessment Report

1 Introduction

The aim of this coursework was to create a custom Network Intrusion Detection System (NIDS) and make it communicate detected attacks (alerts) to a simple SIEM server through a secure communication channel. Finally the alerts must be displayed on screen. Several security functionalities have been implemented: the mutual CHAP authentication, CRC check, timestamp freshness check, HMAC and AES encryption. The content of the message are encrypted with a secret key that in this implementation was hardcoded in the source code. In the next section the workflow of the solution is illustrated.

2 Workflow

The focus during the implementation has been trying to guarantee **Confidentiality, Integrity and Authenticity.**

At the beginning, the first thing to focus on was how to sniff packets and that has been done creating a class (called `sniffer.py`) that uses the *scapy* library to monitor the network traffic on a specific network interface (eth0 or lo, depending on the scope of test). In the class two objects have been injected: a Detection Engine (from `detection_engine.py`) and a Traffic State Tracker (from `traffic_tracker.py`).

2.1 Approach to detection

A quick overview of the initial NIDS pipeline (also shown in the figure 1): the detection architecture follows a sequential mechanism designed to process network traffic in real-time, moving from raw packet capture to stateful analysis and alerting.

The process initiates with the **packet capture** module, which utilizes a sniffer to intercept traffic on the network interface. The system then uses callback mechanism (prn=self.parse_packet) that immediately forwards each captured packet for processing, ensuring low-latency handling of incoming data streams.

This raw data is ingested by the **state aggregation** component, which serves as a feature extraction layer. Here, the system parses headers across the protocol stack (including IP, UDP, TCP, and ARP) to update a set of persistent data structures (dictionaries). Specifically, it maintains counters for packet volume per destination to track potential DoS attacks, uses sets to track unique port and IP associations for identifying horizontal and vertical scanning behaviour, and monitors TCP flag (FPU) combinations to detect specific signature-based anomalies like Christmas Tree packets.

The **detection engine** then evaluates these aggregated metrics against configurable, arbitrary, thresholds (e.g. DoS attack threshold = 100 packets count). It analyses the traffic state maintained by the tracker rather than individual packets. The engine can identify complex behaviours that manifest over time. The logic checks for conditions such as excessive packet counts indicative of flooding, a high cardinality of unique port access requests typical of UDP scanning, and broad ARP queries that suggest network mapping. When a threshold is breached, the system triggers an alert generation process. This phase also incorporates a **cool-down mechanism** to manage alert volume. By checking the timestamp of the last generated notification for a specific attack vector and target, the system won't send redundant alerts within a defined time window. This ensures that the downstream SIEM server receives a concise stream of unique security events rather than a flood of repetitive notifications during high-velocity attacks.

2.2 Secure channel communication

After this, to send the alerts to the SIEM server the NIDS uses a **secure channel** class to communicate (`communication_channel.py`). This is the most crucial step of the process because this is the phase in which the aforementioned requisites must be complied to.

To send data, a custom PDU (Protocol Data Unit) structure is used. Before transmission, every secure PDU is stamped with a timestamp (to prevent replay attacks), given a CRC (for error detection), and signed with an HMAC (for integrity and authenticity).

The process begins with a mutual **CHAP handshake**. The client initiates this with a "hello" message. The only moment packets are not fully wrapped with all security features is during this initial handshake when the challenge nonces are exchanged. However, this is secure because an attacker would need the hardcoded shared secrets to calculate the correct response and complete the authentication; without them, impersonation is impossible. This meets the **Authenticity** requirement.

Once authenticated, the channel enters "Alert Mode". To comply with the **Confidentiality** requirement, the JSON body of every alert PDU is encrypted using AES-256 before being placed into the PDU.

Finally, to maintain a continuous chain of trust, even control messages are secured. Every ACK (Acknowledgement) or NACK sent by the server to confirm receipt is also a fully secure packet: encrypted, timestamped, and HMAC-signed. This ensures that an attacker cannot inject fake acknowledgments to fool the NIDS into thinking an alert was received when it wasn't.

Once the SIEM server receives the alert, it checks it, analyses the content and then it displays the received alert on a CLI. Each alert also gets logged through an instance of the secure logger class (`securelogging.py`) into a file called `siem_alerts.log`. Finally once the communication is completed and the alerts are stored in the log file, the user can also check these logs to see if they have been tampered with the script in `check_logs.py`. This file uses a log verifier class object (`verifylogging.py`), that checks the **Integrity** of the sent alerts, logged with the secure logger that signs the log with an HMAC. Therefore the HMAC is calculated again in the same way (with the same credentials) in order to verify the authenticity of the content.

2.3 NIDS

The NIDS is composed by several components that are present in a sub folder and the class diagram in the figure 2 represents how the elements collaborate to manage the NIDS functionality. Boilerplate security check functions have been aggregated in a separate module (`secure_ops.py`) and in the Pdu class.

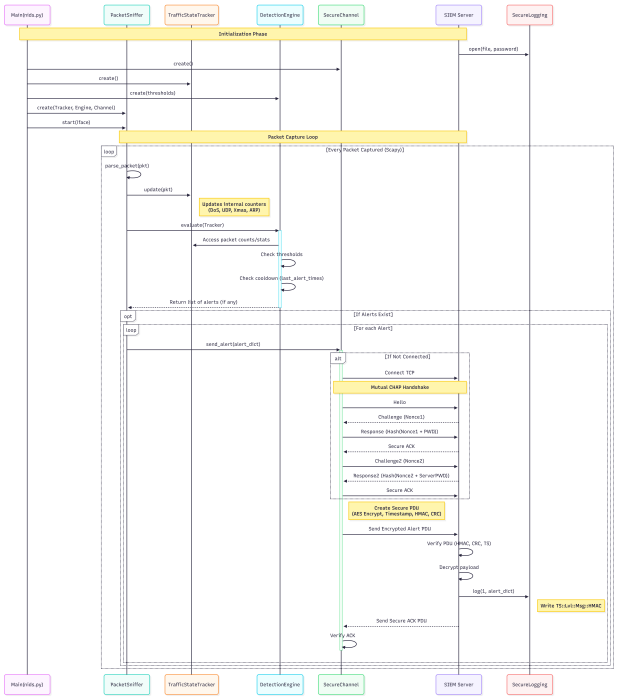


Figure 1: NIDS + SIEM server sequence diagram

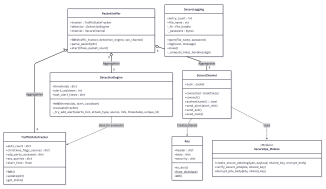


Figure 2: Class diagram

2.4 Logger

The sequence diagram in the figure 3 represents the secure logging mechanism and the following eventual log check.

3 How to run the NIDS

The implementation has been confirmed fully working on the Lancaster University MyLab environment SCC Barrow. **(1)** To run this NIDS, the first thing to do is build the container and then open two terminals. Then in the first terminal you need to run this commands in order to start the SIEM server that will begin listening, waiting for a communication on port 9999 (it is hosted on localhost, so IP 0.0.0.0):

```
connect station
python3 siem_server.py
```

(2) The next step is to open the second terminal and start the NIDS and then select the network interface that needs to be monitored (during the testing phase eth0 has been selected to simulate a more realistic scenario):

```
connect station
```

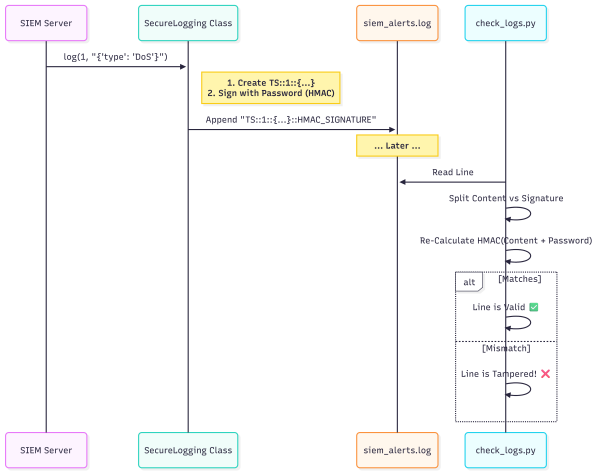


Figure 3: Logger Sequence Diagram

```
python3 nids.py
When it asks which iface to monitor, write "eth0"
```

(3) Now that the SIEM server and the NIDS are running,

```
connect tester
cd test_scripts
python3 udp-port-scan.py 172.20.0.2 9988 9999
python3 udp-dos.py 172.20.0.2 9999 200
python3 arp-scan.py 172.20.0.0/24
python3 tcp-christmas.py 172.20.0.2 9999 2
```

(4) The alerts can be seen when they are being sent from the NIDS on the terminal and they are well displayed and formatted in the SIEM server. Then they are securely logged in the `siem_alert.log` file. To check the integrity of those logs run the following file from one of the station terminals:

```
python3 check_logs.py
```

4 Mitigation for the test scripts

- UDP DoS:
  - Limiting the rate of upcoming UDP packets from a single IP source (at firewall or router level)
  - Udp packets that do not belong to an established connection or known application protocol get dropped
- TCP Christmas Scan:
  - An eventual firewall can be configured to drop any TCP packet with suspicious flag combination. A real TCP stream should never have FIN, URG, and PSH set simultaneously.
- UDP port scan:
  - Keep closed unused ports, therefore close unused services and reduce the attack surface
  - Set up "Honey Ports": whenever these ports are touched send an alert and block the IP source of the scan.
- ARP Scan:

  - Implement static ARP tables in high severity servers, with hard-coded MAC-IP combinations to prevent ARP spoofing attempts or ARP queries.
  - Switch-level security: validate ARP packets against a trusted database.

5 Limitations and possible improvements

The current implementation has several limitations that can be addressed:

- Threshold dependence: The current Detection Engine relies heavily on static thresholds (e.g., "100 packets"). A smart attacker can evade detection by performing "low-and-slow" attacks, sending packets just below the threshold rate to fly under the radar.
- Passive detection: the current implementation only detects intrusions and does nothing to prevent or block the attack (an IPS would do that). Right now it only alerts the SIEM server and the attack continues until human intervention occurs.
- In-memory state: the traffic tracker stores records in python dictionaries that are stored in the RAM. When the NIDS stops or restarts, all the knowledge from previous suspicious behaviour is lost.
- Lack of with-listing: The NIDS pipeline treats all traffic equally. In a real scenario, there should be a whitelist for trusted administrative traffic (e.g., the SIEM server itself) to prevent false positives.

5.1 Improvements

To expand this project to a production-grade tool there is the need to: (i) integrate an IPS in order to achieve an active defence to block the suspicious IPs; (ii) persistent storage to track long term trends and remember attackers after a reboot; (iii) implement an algorithm that adapts the thresholds dynamically (can be Machine Learning, but the dataset to train the model on must be very large to increase accuracy); (iv) avoid hardcoded values, move them into a separate file, abstracting it and giving the possibility to tune the system without interacting with the source code; (v) implement a secure key sharing system (e.g. Diffie-Hellman key exchange) over public channel, possibly deploying it before the mutual CHAP handshake.

A Generative AI Usage Disclosure

Perplexity (Pro Version - Gemini 3 Pro) by Perplexity AI, Inc. (https://www.perplexity.ai) was used for tone and readability feedback, grammar and spelling checks during the writing of the report. It has also been used *while doing the lab exercises* to do debugging, increase code quality and consolidate the code blocks. *Code snippets and functions have been reused* from the labs during the implementation of the NIDS and SIEM server. During the *development of the NIDS and SIEM server*, Perplexity has been used to avoid boilerplate code, consolidate code block, increase readability and debugging,

A.1 Main prompts

Disclaimer: Obviously the integration and reuse of code snippets from the labs exercises brings with it the eventual use of GenAI during the implementation of those (always for aiding, learning and debugging purposes). Moreover, GenAI has been used to aid during the integration of different pieces of code from several different implementations and finally to polish the finished product. The prompts displayed below are related to the coursework and the laboratory exercises.

- `"Help me understand what to do in the Lab 1 (file attached)"`
- `"Help me understand what to do in the Lab 2 (file attached)"`
- `"Help me understand what to do in the Lab 3 (file attached)"`
- `"Help me understand what to do in the Lab 4 (file attached)"`
- `"Help me understand what to do in the Lab 5 (file attached)"`
- `"Help me understand what to do in the Lab 6 (file attached)"`
- `"Whatever message I send I can't see it. Can you explain better the sockets?"`
- `"Explain to me how to use the Scapy library"`
- `"Explain to me how to use Cryptodome in the context of the lab exercise (file attached)"`
- `"Be my mentor, do not correct my code until is not strictly necessary and tell me when you do it. Help me refactor this (file attached)"`
- `"How does the mutual CHAP work"`
- `"What can I improve in my implementation?"`
- `"Can you help me debug this? (file attached)"`
- `"Can you help me integrate this to my current implementation? (files attached)"`
- `"Can you help me refactor the code to reduce boilerplate? (file attached)"`
- `"Why does this error happen? (file attached)"`
- `"What do you think of my implementation? Check it against the coursework requirements (files attached)"`